



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4
по курсу «Языки и методы программирования»
«Iterator и HTTP-сервер на Java»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Интерфейс Iterator	1
HTTP-сервер в Java	1
Реализация Factorize	2
Класс Factorize	3
Реализация HTTP-сервера	5
Парсер параметров запроса	5
JSON-сериализатор	5
Обработчик HTTP	6
Реализация BFS	6
Класс BFS	7
Тестирование	8
Тест Factorize	8
HTTP-сервер	8
Тест BFS	9
Вывод	9

Цель работы

Изучить интерфейс `Iterator` в Java и его реализацию. Создать итераторы для различных структур данных (разложение числа на простые множители, обход графа). Реализовать HTTP-сервер для обработки запросов с передачей данных в формате JSON.

Задачи:

- Реализовать интерфейс `Iterator` для класса `Factorize`.
- Создать HTTP-сервер с использованием `com.sun.net.httpserver`.
- Реализовать JSON-сериализацию для передачи данных.
- Использовать итератор `BFS` для обхода графа в ширину.

Теоретические сведения

Интерфейс `Iterator`

`Iterator` — это интерфейс для последовательного доступа к элементам коллекции.

Основные методы:

- `hasNext()` — проверка наличия следующего элемента
- `next()` — получение следующего элемента
- `remove()` — удаление текущего элемента (опционально)

Классы, реализующие `Iterable`, могут использоваться в цикле `for-each`.

HTTP-сервер в Java

Встроенный HTTP-сервер из пакета `com.sun.net.httpserver` позволяет создавать простые HTTP-серверы без внешних библиотек.

Реализация Factorize

Класс Factorize реализует интерфейс Iterable для разложения числа на простые множители.

Класс Factorize

Factorize.java

```
public class Factorize implements Iterable<Integer> {
    private int number;

    public Factorize(int n) {
        number = n;
    }

    public Iterator<Integer> iterator() {
        return new FactorizeIterator();
    }

    private class FactorizeIterator implements Iterator<Integer> {
        private int current_prime;

        private boolean prime(int x) {
            for (int i = 2; i < x; i++) {
                if (x % i == 0) {
                    return false;
                }
            }
            return true;
        }

        private void findNextPrime() {
            current_prime++;
            for (; current_prime ≤ number; current_prime++) {
                if (number % current_prime == 0 && prime(current_prime)) {
                    return;
                }
            }
        }

        public FactorizeIterator() {
            current_prime = 1;
            findNextPrime();
        }

        public boolean hasNext() {
            return number ≥ current_prime;
        }

        public Integer next() {
            int pow = 0;
            int factor = 1;
            while (number % factor == 0) {
                pow++;
                factor *= current_prime;
            }
            findNextPrime();
        }
    }
}
```

Пример: $5402250 = 2^6 \cdot 3^2 \cdot 5^3 \cdot 7^4$

Реализация HTTP-сервера

Парсер параметров запроса

Server.java

```
class QueryParser {
    static Map<String, String> parse(String query) {
        Map<String, String> params = new HashMap<>();
        if (query == null || query.isEmpty())
            return params;

        for (String pair : query.split("&")) {
            String[] parts = pair.split("=", 2);
            if (parts.length != 2)
                continue;

            String key = URLDecoder.decode(parts[0],
                StandardCharsets.UTF_8);
            String value = URLDecoder.decode(parts[1],
                StandardCharsets.UTF_8);
            params.put(key, value);
        }
        return params;
    }
}
```

JSON-сериализатор

Server.java

```
class JsonSerializer {
    static String toJson(Factorize f) {
        StringBuilder sb = new StringBuilder();
        sb.append("[");

        for (int n : f) {
            sb.append(n);
            sb.append(", ");
        }
        sb.delete(sb.length() - 2, sb.length());

        sb.append("]");
        return sb.toString();
    }
}
```

Обработчик HTTP

Server.java

```
public class Server {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new
        InetSocketAddress(7518), 0);
        server.createContext("/", new MyHandler());
        server.setExecutor(null);
        server.start();
    }

    static class MyHandler implements Handler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String query = exchange.getRequestURI().getQuery();
            Map<String, String> params = QueryParser.parse(query);

            int number = Integer.parseInt(params.get("n"));
            Factorize f = new Factorize(number);

            String json = JsonSerializer.toJson(f);

            exchange.getResponseHeaders().add("Content-Type",
            "application/json");
            exchange.sendResponseHeaders(200, json.getBytes().length);

            try (OutputStream os = exchange.getResponseBody()) {
                os.write(json.getBytes());
            }
        }
    }
}
```

Реализация BFS

Класс BFS реализует обход графа в ширину с использованием итератора.

Класс BFS

BFS.java

```
public class BFS implements Iterable<Integer> {
    private int[][] A;

    public BFS(int[][] A) {
        this.A = A;
    }

    public Iterator<Integer> iterator() {
        return new BFSIterator();
    }

    private class BFSIterator implements Iterator<Integer> {
        private boolean[] visited;
        private Queue<Integer> queue;

        BFSIterator() {
            visited = new boolean[A.length];
            queue = new LinkedList<>();
            queue.add(0);
        }

        public boolean hasNext() {
            return !queue.isEmpty();
        }

        public Integer next() {
            int v = queue.poll();
            visited[v] = true;
            for (int i = 0; i < A[v].length; i++) {
                if (A[v][i] == 1 && !visited[i]) {
                    queue.add(i);
                    visited[i] = true;
                }
            }
            return v;
        }
    }
}
```


Тестирование

Тест Factorize

Test.java

```
public class Test {  
    public static void main(String[] args) {  
        Factorize f = new Factorize(Math.powExact(2, 6) *  
Math.powExact(3, 2) * Math.powExact(5, 6));  
        for (int p : f) {  
            System.out.println(p);  
        }  
    }  
}
```

Вывод: показатели степеней для каждого простого множителя.

HTTP-сервер

terminal

```
$ java Server  
[*] Starting server ...  
Usage http://net1.yss.su:7518/?n=5402250
```

terminal

```
$ curl "http://net1.yss.su:7518/?n=5402250"  
[6, 2, 3, 4]
```

Ответ: показатели степеней $2^6 \cdot 3^2 \cdot 5^3 \cdot 7^4 = 6, 2, 3, 4$

Тест BFS

dop2/Test.java

```
int[][] graph = {
    { 0, 1, 0, 1, 0 },
    { 1, 0, 1, 0, 1 },
    { 0, 1, 0, 0, 0 },
    { 1, 0, 0, 0, 0 },
    { 0, 1, 0, 0, 0 },
};
//      0
//      / \
//     1   3
//    / \
//   2   4

for (int v : new BFS(graph)) {
    System.out.print(v + " ");
}
// Вывод: 0 1 3 2 4
```

Вывод

В ходе лабораторной работы были реализованы:

1. Iterator для Factorize — класс, реализующий интерфейс Iterable, для разложения числа на простые множители
2. HTTP-сервер — сервер на основе com.sun.net.httpserver, обрабатывающий GET-запросы и возвращающий JSON
3. JSON-сериализатор — преобразование данных Factorize в формат JSON
4. Iterator для BFS — обход графа в ширину с использованием очереди

Все реализации используют стандартные интерфейсы Java (Iterable, Iterator), что обеспечивает их совместимость с циклом for-each и другими компонентами Java Collections Framework.